Do Efficient Generative Sampling Choices Transfer Across GPUs?

Training-Free Acceleration of Diffusion and Flow-Matching Transformers

Iason-Daniil Zakynthinos

***Abstract*—Do quality-efficiency conclusions about diffusion and flow-matching sampling remain valid across GPU platforms when training-free accelerations are introduced? This project tests whether the preferred sampling configuration changes at fixed image quality, and measures the cost of transferring choices between devices. Using pretrained DiT and SiT transformers, I compare plain sampling, feature caching, and INT8/FP8 inference on three GPU platforms. The primary outcome is transfer penalty: the extra latency incurred by transferring a source-device choice instead of selecting on the target. Comparisons separate kernel availability from performance on shared configurations. Execution-cost analysis and feature-reuse diagnostics support the explanation of changes in sampling efficiency and quality.**

I. Research Question and Hypotheses

Do quality-efficiency conclusions about diffusion and flow-matching sampling remain valid across GPU platforms when training-free accelerations are introduced?

H1 (transfer baseline). Within each batch size, a device-specific multiplicative latency factor adequately describes plain sampling across platforms. This separable model predicts stable choices; configuration-by-device interactions are the competing explanation. The test concerns practical selection penalties, not merely unequal speedups.

H2 (cost explanation). Caching and low-precision execution introduce interactions that cannot be explained by NFE or operation count alone. A parsimonious model incorporating effective arithmetic throughput, memory traffic, and fixed execution overhead should better predict held-out latency and transferred choices. Interactions may exist without materially changing the preferred configuration; both outcomes will be reported.

H3 (quality mechanism; exploratory). Relative feature-reuse error should predict caching-induced quality degradation better than step count alone, including when evaluated on the other model. No directional claim that flow matching inherently tolerates caching better is assumed: straighter interpolation paths do not guarantee slowly changing hidden features.

II. Motivation and Related Work

Diffusion and flow-matching models generate samples through repeated network evaluations [1], [2]. DiT and SiT provide comparable transformer backbones while differing in their generative formulation [3], [4]. Numerical solvers reduce the number of evaluations [5], [6]; feature caching reduces computation within an evaluation [7], and post-training quantization changes its arithmetic cost [8]. These interventions therefore alter different components of inference cost.

Quality-cost frontiers and caching schedules are already studied [9], and operation counts are known to be imperfect efficiency proxies [10]. Hardware-dependent speedup alone is therefore not the proposed contribution. The unresolved question tested here is whether platform differences change quality-constrained decisions, by how much, and whether inexpensive measurements can explain or predict those changes. DiT and SiT serve as two related workloads; any family-level interpretation is limited to the checkpoints and sampling procedures tested.

III. Methodology

Models and factors. Public ImageNet-256 DiT-XL/2 and SiT-XL/2 checkpoints; no model retraining. Samplers are DDIM and a specified DPM-Solver variant for DiT, and Euler and Heun ODE for SiT. Each uses 8, 16, or 32 solver steps and one of four execution modes: FP16 baseline, FORA-style caching [7], INT8 W8A8, or FP8 W8A8. These are 48 configurations before availability exclusions. Batch sizes 1 and 16 are separate timing conditions. Actual NFE, including solver substages, guidance execution, and cached calls, is logged independently of nominal steps.

Hardware and implementation. An RTX 5080, Colab T4, and GH200 are three tested platforms, not isolated samples of GPU generation. End-to-end smoke tests establish supported kernels before the sweep. The manifest fixes checkpoint, timestep grid, solver order, guidance, labels, seeds, compilation policy, and VAE. Quantization specifies calibration data, scale granularity, excluded layers, accumulation precision, and actual low-precision kernels. Calibration and evaluation seeds are disjoint; fake quantization is not counted as accelerated execution. PyTorch and torchao versions are pinned where possible, with driver, CUDA, host, and remaining stack differences recorded.

Caching policy and probes. Fix cached attention/MLP outputs, refresh interval, initial refresh, and solver-substage behavior before evaluation. On a small independent probe set, compare reused features with full recomputation at the same state and time. Log relative L2 error, magnitude change, and timestep/layer summaries. Probes run outside timed workloads. Relate these diagnostics to the FID change against the corresponding uncached configuration; associations are not treated as proof of causation.

Quality. Generate 2,000 images per configuration for screening, using clean-fid with fixed ImageNet reference statistics [11] and precision/recall [12]. Before confirmation, freeze three FID ceilings spanning the useful pilot range and the selection rules. Retain configurations near the frontier,

including those whose apparent dominance is unresolved. Confirm with three independent 10,000-image runs per retained configuration; report per-run scores and uncertainty rather than pooling unequal sample sizes. Equal counts do not eliminate FID bias [13].

Most quality evaluation uses the RTX 5080. Before reusing these scores elsewhere, regenerate a prespecified subset covering both models and every shared execution mode, including near-threshold cases. Examine numerical deviations and repeated quality estimates. If equivalence within a declared tolerance is unresolved, evaluate the affected target-device candidates directly or exclude those comparisons from transfer claims.

Timing and energy. After warm-up and compilation, randomize configuration order and repeat measurements in multiple sessions. Report median and interquartile range for synchronized end-to-end generation, with denoising and VAE decoding also timed separately. Distinguish batch latency from throughput and amortized time per image. Record peak memory, power limits, and clocks. Energy is secondary: measure repeated workloads over long windows using supported NVML counters, state the power domain and idle treatment, and omit incomparable measurements.

IV. Analysis, Feasibility, and Expected Contribution

Primary outcome. For each ordered device pair, batch size, and FID ceiling, restrict selection to configurations executable on both platforms and meeting the quality requirement. Choose the fastest on the source and evaluate its target latency. Transfer penalty is target latency divided by the fastest eligible target latency, minus one. Define near-optimal as a penalty of at most 5%. Unsupported methods and empty feasible sets are reported separately. Propagate timing and quality uncertainty; do not force a winner when eligibility or dominance is unresolved.

H1/H2. Compare the common-scaling baseline with configuration-by-device interactions. Fit NFE-only and operation-count baselines, then a small hardware-aware cost model using compute, memory, and overhead terms. Effective rates come from independent representative kernel probes; memory traffic is estimated with stated assumptions. Validate by holding out configuration groups and, exploratorily, one device at a time, without fitting to its end-to-end timings. Report latency error and decision penalty, including failures. Pareto frontiers and latency-only Kendall tau-b on shared configurations are secondary summaries; neither substitutes for the primary outcome.

H3. Test whether feature-reuse diagnostics predict the quality change beyond model, solver, and step count. Evaluate cross-model prediction separately from within-model fit. Report negative results and uncertainty. Differences between DiT and SiT do not identify the causal effect of training objective.

Feasibility. Pilot throughput determines the measured generation budget. The confirmation shortlist and all calibration, timing, and evaluation costs are logged explicitly. The full factor design is retained; optional analyses are distinguished from the primary transfer experiment.

Expected contribution. Evidence on whether the preferred model-sampler-acceleration configuration changes across platforms at fixed image quality; transfer penalties with uncertainty; and an open harness and manifest. Execution-cost and feature-reuse analyses explain the observed changes. Stable choices with tightly bounded penalties are informative. Claims concern the tested platforms and configuration space, not universal GPU behavior. Serving, training, and exhaustive solver or cache-policy optimization remain out of scope.

V. Proposed Schedule

Weeks 1-2: checkpoints, environments, complete kernel smoke tests, and manifest. *3-4:* solver/caching validation, quantization calibration, and timing protocol. *5-7:* quality pilot, feature probes, frozen thresholds and shortlist. *8-9:* cross-platform timing, kernel probes, and quality checks. *10-11:* independent quality confirmation and targeted target-device evaluation. *12-13:* held-out analysis, uncertainty, plots, and report.

References

[1] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Proc. NeurIPS*, 2020.

[2] Y. Lipman et al., “Flow matching for generative modeling,” in *Proc. ICLR*, 2023.

[3] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *Proc. ICCV*, 2023.

[4] N. Ma et al., “SiT: Exploring flow and diffusion-based generative models with scalable interpolant transformers,” in *Proc. ECCV*, 2024.

[5] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *Proc. ICLR*, 2021.

[6] C. Lu et al., “DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps,” in *Proc. NeurIPS*, 2022.

[7] P. Selvaraju et al., “FORA: Fast-forward caching in diffusion transformer acceleration,” arXiv:2407.01425, 2024.

[8] J. Wu et al., “PTQ4DiT: Post-training quantization for diffusion transformers,” in *Proc. NeurIPS*, 2024.

[9] A. Aggarwal, A. Shrivastava, and M. Gwilliam, “Evolutionary caching to accelerate your off-the-shelf diffusion model,” in *Proc. ICLR*, 2026.

[10] M. Dehghani et al., “The efficiency misnomer,” in *Proc. ICLR*, 2022.

[11] G. Parmar, R. Zhang, and J.-Y. Zhu, “On aliased resizing and surprising subtleties in GAN evaluation,” in *Proc. CVPR*, 2022.

[12] T. Kynkäänniemi et al., “Improved precision and recall metric for assessing generative models,” in *Proc. NeurIPS*, 2019.

[13] M. J. Chong and D. Forsyth, “Effectively unbiased FID and Inception Score and where to find them,” in *Proc. CVPR*, 2020.